

**CONFIDENTIAL**



**UTM** | Faculty of  
Computing  
UNIVERSITI TEKNOLOGI MALAYSIA

**FINAL EXAMINATION SEMESTER I, 2022/2023**

**PAPER 1 / THEORY**

**SUBJECT CODE** : **SECJ1023 / SCSJ1023**

**SUBJECT NAME** : **PROGRAMMING TECHNIQUE II**

**SECTION** : **1 - 11 and 15**

**TIME / DURATION** : **\_\_\_ / 2 HOURS**

**DATE/DAY** : **\_\_\_ 2023 / \_\_\_ DAY**

**VENUE** : **\_\_\_**

---

**INSTRUCTIONS :**

This test book consists of TWO (2) sections:

Section A: Multiple Choice Questions 30 Marks

Section B: Structured Questions 70 Marks

**TOTAL 100 Marks**

**ANSWER ALL QUESTIONS IN THE ANSWER BOOKLET.**

The question test book must be returned at the end of the test.

This question paper consists of 15 printed pages excluding this page.

**SECTION A: MULTIPLE CHOICES SINGLE ANSWER****[30 Marks]**

Instruction: Read each question below carefully and choose the letter A, B, C, or D that **BEST** describes the answer. Write your answer in the answer booklet. Each question carries **1.5 marks**.

1. In the given scenario, the class Library has a collection of books, but the books can exist independently and can be associated with multiple libraries.

What type of relationship exists between the class Library and the class Book ?

A. Inheritance

B. Compositions

☒ C. Aggregation → "has-a" relationship, can exist without parent

D. Encapsulation

2. Determine the relationship/s is/are implemented in the following code segment.

```
class Department {  
    // details codes are left out for brevity  
};  
  
class College {  
    // details codes are left out for brevity  
};  
  
class Student {  
    // details codes are left out for brevity  
    Department dept; → composition, nested object  
    College *college; → pointer, aggregation  
};
```

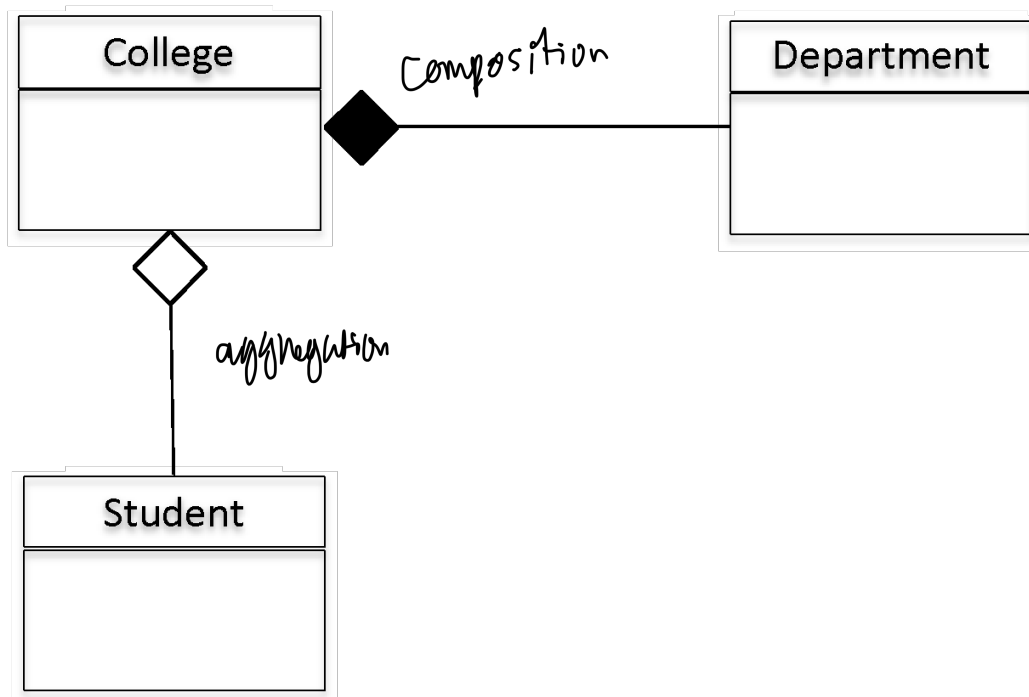
☒ A. Aggregation and composition

B. Aggregation only

C. Composition only

D. No relationship

3. Based on the following class diagram, which type of relationship/s is/are implemented in the diagram?



- ☒ A. Aggregation and composition
- B. Association and aggregation
- C. Association and composition
- D. Composition only
4. Which of the following is **FALSE** about the association?
- A. Association indicates the relationships between classes through their objects. ✓
- ☒ B. The association can be one-to-one and many-to-many relationships only. ✗
- C. The association can be one-to-one and one-to-many relationships. ✓
- D. Association allows one object instance to cause another to perform and then act on its behalf. ✓

```

1#include <iostream>
2#include <string>
3using namespace std;
4
5class Car {
6    private:
7        string make, model;
8    public:
9        Car(string make, string model) {
10            this->make = make;
11            this->model = model; }
12        void print(){ cout << make << "-" << model; } };
13
14class Person {
15    private:
16        Car* car;
17        string name;
18        int year;
19    public:
20        Person(string name, int year) {
21            this->name = name;
22            this->year = year; }
23        void buyCar(Car* c){ car = c; }
24        void sellCar(){ car = NULL; }
25        void print(){
26            cout << name << "-" << year << ": ";
27            if (car!= NULL) car->print();
28            cout << endl; } };
29
30int main() {
31    Person person1("Jack", 2023);
32    Car *car1 = new Car("Proton", "X70");
33    person1.buyCar(car1);
34    person1.print();
35    person1.sellCar();
36    person1.print();
37    return 0; }

```

aggregation

5. Consider the given program above. Which of the following statements is **FALSE** about the program ?

- A. Line 16 implements the aggregation concept. ✓
- ☒ B. Line 16 implements the composition concept. ✗
- C. A person may have a car. ✓
- D. An object of Person may be created without an object of Car. ✓

6. Determine the result of the following program?

```

1 using namespace std;
2
3 class Base1 {
4     public:
5         Base1()
6         { cout << " Base1" << endl; }
7 };
8
9 class Base2 {
10     public:
11         Base2()
12         { cout << "Base2" << endl; }
13 };
14
15 class Derived: public Base1, public Base2 {
16     public:
17         Derived()
18         { cout << "Derived" << endl; }
19 };
20
21 int main() {
22     Derived d;
23     return 0;
24 }

```

*will call base class constructor first, then child constructor last*

*depends on sequence*

A. A compilation error occurs.

☒ B. Base1 Base2 Derived

C. Base2 Base1 Derived

D. A run-time error occurs.

7. If a base class is inherited with protected access mode, which of the following statements is **TRUE**?

☒ A. Public and Protected members of the base class become protected members of the derived class.

B. Only protected members become protected members of the derived class.

C. Only private members of the base class become private of the derived class.

D. All private, protected, and public members of the base class become private of the derived class.

*public member → protect  
protected → protect  
private → private*

*class Derived : protected Base*

```

1  #include <iostream>
2  using namespace std;
3
4  class Animal {
5      public:
6          Animal()
7          { cout << "Animal constructor executing" << endl;
8          }
9
10         ~Animal()
11         { cout << "Animal destructor executing." << endl;
12         }
13     };
14
15     class Cat : public Animal {
16     public:
17         Cat() : Animal()
18         { cout << "Cat constructor executing" << endl; }
19
20         ~Cat()
21         { cout << "Cat destructor executing" << endl; }
22     };
23
24     int main() {
25         Animal *myAnimal = new Cat();
26
27         delete myAnimal;
28
29         return 0;
30     }

```

*Handwritten notes:*

- create an object of Cat, but cat inherit from animal, it must first invoke animal constructor
- only Animal destructor, because the pointer type is Animal - because there is no virtual in the base class, static binding, only base class destructor

8. Consider the program above. Which of the following statements is **TRUE** about the program?

- ☒ A. In Line 25, both constructors in class Animal and Cat will execute.?
- B. In Line 27, both destructors in class Animal and Cat will execute.
- C. In Line 25, only constructors in class Animal will execute.
- D. In Line 27, only destructors in class Cat will execute.

9. What is the purpose of Inheritance of object-oriented programming?

- A. To establish a "has a" relationship between classes. *is a* ~~has a~~ is for composition / Aggregation
- ☒ B. To define a relationship between parent and child classes.
- C. To encapsulate data and methods within a class. ~~encapsulation~~
- D. To create an instance of a class.

```

Class Base{
    private: x;
    protected: y;
    public: z;
};

Class Derived: private
Base{};

```

10. Consider the class declaration above. Which of the following statements is **TRUE** about how the inherited base class members appear in the class Derived?

- A. x is inaccessible, y is private, and z is protected.
- ☒ B. x is inaccessible, y is private, and z is private. ✓
- C. x is inaccessible, y is protected, and z is public.
- D. x is protected, y is public, and z is public. ✗

11. Which of the following is the **CORRECT** syntax to declare a Pure Virtual Function?

- ☒ A. virtual void display()=0;
- B. virtual void display ()==0;
- C. void virtual display ()==0;
- D. void display virtual ()=0;

12. Which one of the following statements is TRUE about a pure virtual method? *must be parent*

- ☒ A. A pure virtual method is a method in a parent class declared as virtual without any definition
- B. A pure virtual method is a method in a parent class declared as virtual with at least one definition
- C. A pure virtual method is a method in a child class declared as virtual with at least one definition
- D. A pure virtual method is used to implement the polymorphism concept.

\* abstract class → any class that contains at least one pure virtual function and cannot be instantiated

13. Identify abstract class/es in the following program.

```
class Person{
    string name;
public:
    Person(string n):name(n){}
    virtual void print() const = 0;
};

class Lecturer : public Person{
    string department;
public:
    Lecturer(string d):department(d){}
};

class Student : public Person{
    string matric;
public:
    Student(string m):matric(m){}
    void print() const {cout << matric;}
};

class PostGraduate : public Student{
    string research;
public:
    Student(string r):research(r){}
    virtual void setProject() = 0;
};
```

PVP  
- abstract

inherited from class that  
has PVP  
- does not override a virtual  
function  
- abstract

overwrite print(), which is  
virtual.  
- not abstract

abstract

I - Person /

II - Lecturer /

III - PostGraduate /

IV - Student

A. Only I

B. I and II

**C. I, II, and III**

D. I, II, III, and IV.

14. Which of the following concepts **DOES NOT APPLY** to the following program?

```
1  #include <iostream>
2  using namespace std;
3
4  class Calculation{
5  private:
6      int num1, num2;
7  public:
8      Calculation():num1(0), num2(0) {}
9      Calculation(int n1, int n2):num1(n1), num2(n2) {}
10     int getNum1() { return num1; }
11     int getNum2() { return num2; }
12     virtual void display() {
13         cout << num1 << "&" << num2 << ": "; }
14 };
15
16 class Addition : public Calculation {
17 private:
18     int sum;
19 public:
20     Addition() { sum = 0; }
21     Addition(Calculation *c) {
22         sum = c->getNum1() + c->getNum2(); }
23     void display() { cout << sum << "; "; }
24 };
25
26 class Multiplication : public Calculation{
27 private:
28     int product;
29 public:
30     Multiplication() { product = 0; }
31     Multiplication(Calculation *c) {
32         product = c->getNum1() * c->getNum2(); }
33     void display() { cout << product; }
34 };
35
36 int main(){
37     Calculation* p = new Calculation(10,23);
38     Addition* a = new Addition(p);
39     Multiplication* m =new Multiplication(p);
40     Calculation* s[3] = {p, a, m};
41     for(int i = 0; i < 3; i++) { s[i]->display(); }
42     return 0;
43 }
```

A. Virtual method ✓

☒ B. Abstract class ✗

C. Dynamic binding ✓

D. Overriden method ✓

15. Consider the program below. Which part of the program implements the concept of polymorphism?

```

1  Class Shape {
2      public:
3          virtual string getPerimeter(){return 0;}
4          int getType() {return 0;}
5  };
6  Class Rectangle : public Shape {
7      public:
8          double getPerimeter() { return 2 * (l + w); }
9          int getType() {return 1;}
10
11 };
12
13
14 class Circle : public Shape {
15     public:
16         void getPerimeter() {return 2 * 3.142 * r; }
17         int getType() {return 2;}
18 };

```

- A. Class Shape
- B. Class Rectangle and Class Circle
- ☒ C. Method getPerimeter()
- D. Method getType()

16. Based on the program below, which of the following statements is **TRUE** about the try and catch?

```

1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      try {
6          throw 'a';
7      }
8      catch (int x) {
9          cout << "Caught ";
10     }
11     return 0;
12 }

```

*Handwritten notes:*

- a char (pointing to 'a')
- but this only catch integer (pointing to catch block)
- so exception is not handled here (pointing to catch block)
- program will terminate (pointing to catch block)
- catch block (pointing to catch block)

- A. The code in the try-block is called an exception handler.
- ☒ B. The code in the try-block is called protected code.
- C. The code in the catch-block is called protected code.

- D. If no catch block that matches the exception is found, the program will keep on executing up to infinite. *→ just will terminate*

17. Which of the following statements is **TRUE** about the program below?

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 template <typename T>
6
7 class Data{
8 private:
9     T data;
10 public:
11     Data(T value) : data(value) {}
12     void print() { cout << data << " "; }
13 };
14
15 int main() {
16     Data<const char*> c("good luck");
17     Data<int> t(2);
18     Data<string> s("final exam");
19     c.print(); → this would print "good luck" not just "g"
20     t.print();
21     s.print();
22     return 0;
23 }
```

- A. The output of this program is as follows: g 2 final exam.
- (B)** T is the type of Data template that replaces char, int, and string.
- C. The template is the keyword to influence the data type.
- D. In Line 9, T is the class, and the data is the object to relate to the template.

*type parameter*

*member variable*

18. Choose a statement that is **TRUE** about error handling with exceptions.

- (A)** Error handling is done by allowing the error to occur then it is handled by catching it later.
- B. Error handling is done by anticipating an error before it happens.
- C. Error handling is done by safeguarding code with the if block to prevent any error to occur.
- D. Error handling is done by ignoring the execution of code that may raise errors.

19. What is the output for the following program if the input value is **120 120.55**?

```
1#include <iostream>
2#include <iomanip>
3using namespace std;
4
5template <class T>
6T dividetwo(T number) {
7    return number/2;
8}
9
10int main() {
11    int ival;
12    double dval;
13
14    cout << setprecision(3);
15    cout << "Enter an integer and a double value: ";
16    cin >> ival >> dval;
17    cout << "Value divided by two: ";
18    cout << dividetwo(ival) << " and "
19         << dividetwo(dval) << endl;
20    return 0;
21}
```

- ☒ A. Value divided by two: 60 and 60.3
- B. Value divided by two: 60 and 60.0
- C. Value divided by two: 60.3 and 60.3
- D. Value divided by two: 60.0 and 60.0

20. Which of the following statements is **TRUE** about vectors and arrays?

- ☒ A. Vectors have a dynamic size whereas arrays have a static size.
- B. Both vectors and arrays have a dynamic size.
- C. Both vectors and arrays have a static size.
- D. Vectors have a static size whereas arrays have a dynamic size.

array → size can't change  
vector → can grow or shrink when program is running

## SECTION B: STRUCTURED QUESTIONS

[70 Marks]

**Instruction:** This section consists of THREE (3) questions. Answer all questions. The marks are as indicated in the question.

### Question 1

(30 Marks)

- a. Describe the difference between aggregation and composition.

Provide examples for each type of relationship.

(6 marks)

- b. Discuss the following programming concepts and for each of them, give an example use case or scenario where the concept is applicable.

i inheritance

ii polymorphism

iii abstract class

iv class template

(20 marks)

- c. Explain with an appropriate example, how map is useful in C++ programming.

(4 marks)

a. Aggregation: This is a "weak" association or a "has-a" relationship where the child object can exist independently of the parent. If the parent object is destroyed, the child object continues to exist.

example:

A library and its Book, if a library is closed or destroyed, the books themselves still exist and can be moved to another library.

composition:

This is a "strong" association where the child object is dependent on the parent; the child cannot exist without the parent. If parent object is destroyed, the child objects associated with it are destroyed as well.

example:

college and its department. If a college ceases to exist, the department within that specific organizational structure also ceases to exist.

b) i) inheritance

- mechanism where a new (derived/child) class derives properties and behaviours from an existing class (base/parent). It enables code reusability.

use case:

creating a base class vehicle with properties like brand and methods like accelerate(), and having child class Car and Motorcycle inherit these traits while adding their own specific features.

ii) Polymorphism

- is the ability of object to perform the same action differently.
- allows object of different class to be treated as object of base class.

use case:

having a base class shape with a method draw(). When draw() is called on a circle or rectangle object, each performs its own specific drawing logic.

iii) Abstract Class:

A class that contains at least one pure virtual function. It cannot be instantiated and serves as a template for derived classes.

use case:

A Person class that declares virtual void print() = 0; . You cannot have a generic "Person", but you can have specific "Student" or "Lecturer" objects that implement the printing function.

iv) Class Template

- A powerful feature that allows you to write a single class definition that can work with any data type, making the code generic.

use case:

Creating a stack<T> or Data<T> class where T can be int, string or any other type, allowing the same logic to manage different types of data.

c) A std::map is an associative container that stores elements formed by a combination of a key value and a mapped value.

- useful because it allows for efficient data retrieval based on a unique key, similar to a dictionary.

Example → You can use a map<string, int> to store student names (key) and their corresponding marks (values). This allows to quickly lookup a student's mark by their name without searching through the entire list linearly.

## Question 2

(20 Marks)

Consider the following program.

```
1 class Engine {
2     protected:
3         double horsepower;
4 };
5
6 class Wheel {
7     protected:
8         double size;
9 };
10
11 class Radio {
12     protected:
13         string brand;
14 };
15
16 class Vehicle {
17     protected:
18         string manufacturer;
19         Wheel wheel;
20 };
21
22 class Bicycle : public Vehicle {
23     protected:
24         bool hasSidecar;
25 };
26
27 class SportsCar : public Vehicle, public Engine {
28     protected:
29         bool isConvertible;
30 };
```

- a. Based on the program, determine whether the following classes form any relationship in terms of Object-Oriented Programming. If so, write the name of the relationship, otherwise write “No relationship”.

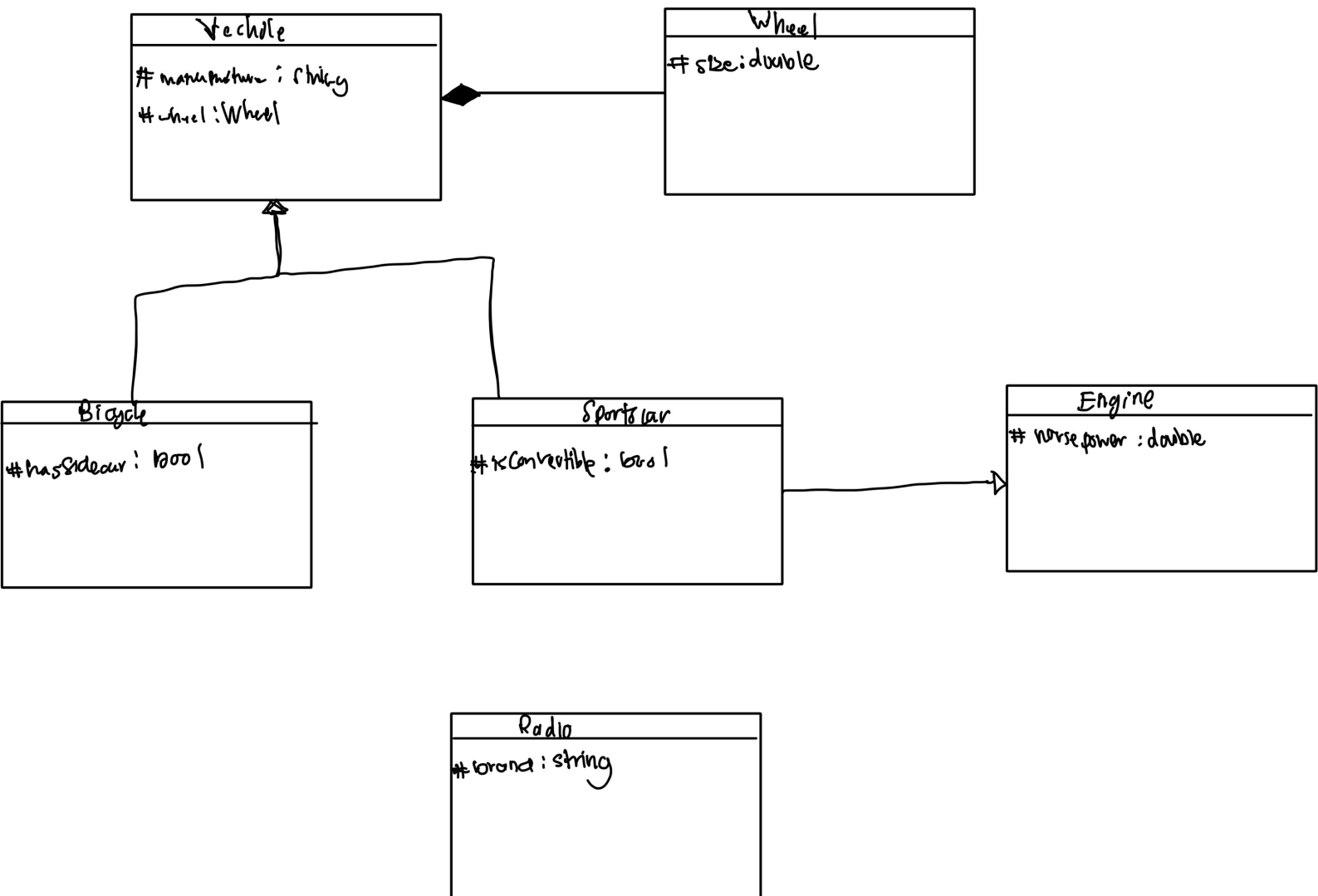
- i. Vehicle and Bicycle *inheritance*
- ii. Wheel and Vehicle *composition*
- iii. Vehicle and Engine *no relationship*
- iv. Bicycle and Sportscar *no relationship*
- v. Sportscar and Vehicle *inheritance*

(10 marks)

- b. Draw the class diagram for the program.

(10 marks)

b.



**Question 3****(20 Marks)**

Answer the following questions based on the given program.

```
1  #include<iostream>
2  #include<string.h>
3  #include<stdio.h>
4  using namespace std;
5
6  class publisher {
7      char pname[15];
8      char hoffice[15];
9      char address[25];
10     double turnover;
11
12     protected:
13         char phone[3][10];
14         void register();
15
16     public:
17         publisher();
18         ~publisher();
19         void enter data();
20         void disp data();
21 };
22
23 class branch {
24     char bcity[15];
25     char baddress[25];
26
27     protected:
28         int no_of_emp;
29
30     public:
31         char bphone[2][10];
32         branch();
33         ~branch();
34         void have data();
35         void give data();
36 };
37
38 class author : public branch, publisher {
39     int aut_code;
40     char aname[20];
41     float income;
42
43     public:
44         author();
45         ~author();
46         void getdata();
47         void putdata();
48 };
```

has no access specifier  
- so it become private

a. Which type of inheritance is implemented in the program?

multiple inheritance

(1 mark)

b. Name the base class(/es) and derived class (/es).

base → publisher, branch

derived → author

(3 marks)

c. Give the sequence of Constructor/Destructor Invocation when object of class author is created.

(6 marks)

d. Which data members are accessible from the objects of class author.

(5 marks)

e. Which member functions are accessible from the object of class author.

(5 marks)

c) constructor → 1) branch()  
2) publisher()  
3) author()

d) bphone

destructor → 1) ~author()  
2) ~publisher()  
3) ~branch()

reverse of  
constructor order

from author :

1) void get\_data()  
2) void put\_data()

from branch :

1) void have\_data()  
2) void give\_data()